

PEX0130 - Programação Orientada a Objetos

Polimorfismo e Interfaces

Bruno Borges da Silva
bruno.silva@temporarios.ufersa.edu.br

Introdução POLIMORFISMO

- ◆ O Polimorfismo é uma **técnica** para lidar com a **complexidade** dos softwares.
- ◆ Ele nos permite escrever programas que trata uma ampla variedade de classes relacionadas existentes e ainda que serão especificadas.
- ◆ Ou seja, é possível **projetar e implementar** sistemas mais flexíveis e extensíveis.
- ◆ Trata as “várias formas” de um objeto.
- ◆ Utiliza a **herança** para tal tarefa.

Introdução

- ◆ Quando utilizamos a herança é possível criar objetos da seguinte forma:
 - Referência a um objeto superclasse com outra superclasse.
 - EX: `Funcionario f = new Funcionario();`
 - Referência a um objeto subclasse com outra subclasse.
 - Ex: `Professor p = new Professor();`
 - Referência a um objeto superclasse com outra subclasse.
 - Ex: `Funcionario p = new Professor();`

Relação superclasse - subclasse

- ◆ É possível e seguro fazer uma referência a um objeto da superclasse com uma referência da subclasse.
- ◆ Isso é possível porque o objeto da subclasse “é um” objeto de sua superclasse.
- ◆ Quando criamos um objetos com essa relação, tratamos o objeto criado como um objeto da superclasse, porém, se o código de um método for sobrescrito, o código executado será o da subclasse.
- ◆ `Funcionario p = new Professor();`

Polimorfismo

- ◆ Exemplo de aplicação de Polimorfismo na prática.
 - Classe Funcionário.

```
public class Funcionario {  
  
    private String nome;  
    private double salario;  
  
    public double ganhoAnual() {  
        double ganho = this.salario * 12;  
        return ganho;  
    }  
  
    public void exhibeDados() {  
        System.out.println("Nome: " + nome  
            + " Salário: " + salario);  
    }  
}
```

Polimorfismo

- ◆ Exemplo de aplicação de Polimorfismo na prática.
 - Classe Técnico que herda de Funcionário.

```
public class Tecnico extends Funcionario{  
  
    private double bonus = 100;  
  
    public double ganhoAnual() {  
        double ganho = (super.getSalario()+bonus)*12;  
        return ganho;  
    }  
}
```

Polimorfismo

- ◆ Exemplo de aplicação de Polimorfismo na prática.
 - Instâncias de Técnico e Funcionário.

```
public class TesteFuncionario {  
  
    public static void main(String[] args) {  
        Funcionario f = new Tecnico();  
        f.setNome("Nickerson");  
        f.setSalario(1000);  
        f.exibeDados();  
        System.out.println(f.ganhoAnual());  
  
        Funcionario f2 = new Funcionario();  
        f2.setSalario(1000);  
        System.out.println(f2.ganhoAnual());  
    }  
}
```

Polimorfismo

- ◆ Exemplo de aplicação de Polimorfismo na prática.
 - Instâncias de Técnico e Funcionário.
 - Saídas de ganho anual de ambas as instâncias.
 - Método ganhoAnual().

```
run:
```

```
Nome: Nickerson Salário: 1000.0
```

```
13200.0
```

```
12000.0
```

Classes Abstratas

- ◆ Podem existir casos em que a classe se comporta como um tipo, logo, supomos que os objetos desse tipo não serão instanciados.
- ◆ Nesses casos, chamamos a classe de classe Abstrata.
- ◆ O único objetivo dessa classe é servir de superclasse para outras classes.
- ◆ As classes que herdam de classes abstratas são conhecidas como classes concretas.
- ◆ Temos uma característica exclusiva desse tipo de classe, os métodos abstratos.

Classes Abstratas

- ◆ Classe Abstrata na prática.
 - Classe Forma com métodos abstratos: desenhar() e informações().

```
public abstract class Forma {  
  
    public abstract void desenhar();  
  
    public abstract void informacoes();  
  
    public void teste() {  
        System.out.println("Testando!!");  
    }  
}
```

Exercícios - POLIMORFISMO

- ◆ Criar uma superclasse chamada animal e as 3 seguintes subclasses: cachorro, cavalo e preguica. Segue as classes com seus respectivos atributos e métodos.
- ◆ Classe abstrata Animal possui um nome e uma idade e um método abstrato emitirSom.
- ◆ Classe Cachorro herda de Animal e sobreescreve o método emitirSom.
- ◆ Classe Cavalo herda de Animal e sobreescreve o método emitirSom.
- ◆ Classe Preguica herda de Animal e sobreescreve o método emitirSom.

Exercícios - POLIMORFISMO

- ◆ Classe TesteAnimais que tem um vetor de Animal com 10 posições.
 - Coloque um Animal em cada posição.
 - Depois percorra o vetor emitindo o som de cada Animal.

Introdução INTERFACES

- ◆ As vezes necessitamos que um conjunto de classes possuam os mesmos comportamentos, porém não sejam filhos de uma super classe.
- ◆ Para isso podemos criar uma interface.
- ◆ Interface funciona como um contrato, informando que as classes que implementarem essa(s) interface(s) possuem comportamentos semelhantes.
- ◆ Funciona de forma semelhante às Classes Abstratas, porém não possuem o relacionamento “é um”.

Introdução INTERFACES

◆ Exemplo de interface.

```
1 package javaObjectOriented.heranca2polimorfismo;
2
3 public interface Iinterface { no usages new *
4     // Constantes (implicitamente public, static e final)
5     String CONSTANTE = "valor"; no usages
6
7     // Métodos default
8     default void metodoDefault() { no usages new *
9     }
10
11     // Métodos estáticos
12     static void metodoEstatico() { no usages new *
13     }
14
15     // Métodos privados
16     private void metodoPrivado() { no usages new *
17     }
18 }
19
```

Introdução INTERFACES

- ◆ Implementando uma interfaces em uma classe.

```
1 package javaObjectOriented.heranca2polimorfismo;
2
3 public class ClasseImplementada implements Iinterface {
4     @Override no usages new *
5     public void metodoAbstrato() {
6         System.out.println("Abstrato");
7     }
8 }
9
```

Introdução INTERFACES

- ◆ Diferente de herança (extends), uma classe pode implementar múltiplas interfaces.

```
package javaObjectOriented.heranca2polimorfismo.familia;
```

```
public interface Mae { 1 usage 1 implementation new *  
    default void habilidadeMae() { 1 usage new *  
        System.out.println("Habilidades da MAE ...");  
    }  
}
```

```
package javaObjectOriented.heranca2polimorfismo.familia;
```

```
public interface Pai { 1 usage 1 implementation new *  
    default void habilidadePai() { 1 usage new *  
        System.out.println("Habilidades dp PAI ...");  
    }  
}
```

Introdução INTERFACES

- ◆ Diferente de herança (extends), uma classe pode implementar múltiplas interfaces.

```
public class Filho implements Pai, Mae { 2 usages new *  
    // ...  
    private String nome; 4 usages  
  
    public Filho(String nome) { this.nome = nome; }  
  
    public String getNome(boolean showName) { 1 usage new *  
        if (showName) {  
            System.out.printf("Meu nome é %s\n", this.nome);  
        }  
        return nome;  
    }  
  
    public String getNome() { new *  
        return nome;  
    }  
}
```

Introdução INTERFACES

- ◆ Diferente de herança (extends), uma classe pode implementar múltiplas interfaces.

```
public class Main { new *
    public static void main(String[] args) { new *
        Filho filho = new Filho(nome: "Bruno");

        filho.habilidadeMae();
        filho.habilidadePai();
        filho.getNome(showName: true);

        System.out.println("Meu nome é " + filho.getNome());
    }
}
```

Exercícios - INTERFACES

- ◆ Crie a seguinte hierarquia de classes:
- ◆ Uma interface para representar qualquer forma geométrica, definindo métodos para cálculo do perímetro e cálculo da área da forma;
- ◆ Uma classe abstrata para representar quadriláteros. Seu construtor deve receber os tamanhos dos 4 lados e o método de cálculo do perímetro já pode ser implementado;
- ◆ Classes para representar retângulos e quadrados. A primeira deve receber o tamanho da base e da altura no construtor, enquanto a segunda deve receber apenas o tamanho do lado;
- ◆ Uma classe para representar um círculo. Seu construtor deve receber o tamanho do raio.

Exercícios - INTERFACES

- ◆ No programa principal, pergunte ao usuário quantas formas ele deseja criar. Em seguida, para cada forma, pergunte se deseja criar um quadrado, um retângulo ou um círculo, solicitando os dados necessários para criar a forma. Todas as formas criadas devem ser armazenadas em um vetor. Finalmente, imprima:
 - (a) os perímetros; e
 - (b) as áreas de todas as formas.